

Tiny Vision-Language Model (Tiny VLM)

Technical Report & Experimental Evaluation — RLS Entrance Challenge

Aya Smairi

Research Lab SUP'COM (RLS)

September 28, 2026

Abstract

This technical report presents the end-to-end design, implementation, and empirical evaluation of a custom lightweight Vision-Language Model (Tiny VLM) developed from scratch for the Research Lab SUP'COM (RLS) entrance challenge. The architecture integrates a Convolutional Neural Network (CNN) visual encoder, a linear projection adapter, and an autoregressive Transformer decoder with a handwritten Multi-Head Attention mechanism. System optimization was verified via an overfitting sanity test ($E0$), achieving near-zero loss ($\mathcal{L}_{CE} \approx 0.0035$). Evaluated on the *ShapeScenes* test set, the baseline model achieves an **Exact Match accuracy of 59.7%**. An image-blind ablation study ($E1$) reduces performance to **1.8%**, confirming genuine visual grounding. Finally, throughput scaling benchmarks ($S1$) demonstrate a **47×** speedup on GPU hardware (NVIDIA Tesla T4 at **2459.8 images/sec** with batch size 64) compared to CPU execution.

1 Introduction & Challenge Scope

Vision-Language Models (VLMs) combine visual perception with language generation. The goal of the RLS entrance challenge is to design a compact VLM capable of reading synthesized 64×64 RGB scene images and generating character-by-character textual descriptions detailing geometric object attributes and spatial relations (e.g., predicting "**largeredcircle**").

To demonstrate deep architectural understanding, the entire pipeline was constructed manually without using PyTorch's high-level attention abstractions.

2 Dataset & Task Definition

The *ShapeScenes* dataset contains synthetic images featuring geometric objects characterized by size, color, shape, and spatial relationships. Target strings are tokenized at the character level, padded to a uniform sequence length using `<PAD>` tokens, and terminated with an end-of-sequence token (`<EOS>`).

3 Model Architecture

3.1 CNN Visual Encoder & Adapter Projection

The visual encoder processes an input image $X \in \mathbb{R}^{B \times 3 \times 64 \times 64}$. A series of convolutional layers extracts spatial feature maps, which are flattened and projected linearly into the model’s hidden dimension d_{model} to form a sequence of N_v visual tokens:

$$Z_v = \text{Linear}(\text{Flatten}(\text{CNN}(X))) \in \mathbb{R}^{B \times N_v \times d_{\text{model}}} \quad (1)$$

3.2 Custom Multi-Head Attention (MHA)

The Multi-Head Attention module was implemented from scratch without relying on `torch.nn.MultiheadAttention`. Given projected Query, Key, and Value matrices $Q, K, V \in \mathbb{R}^{B \times N \times d_{\text{model}}}$, scaled dot-product attention across h heads of dimension $d_k = d_{\text{model}}/h$ is formulated as:

$$\text{Attention}(Q_i, K_i, V_i) = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} + M \right) V_i \quad (2)$$

where M is the combined attention mask comprising:

- **Causal Mask:** A lower-triangular mask ensuring token position t cannot attend to future tokens $t' > t$.
- **Padding Mask:** Zeroes out attention weights for padded sequence positions.

3.3 Autoregressive Decoder & Greedy Inference

The Transformer decoder processes visual tokens Z_v concatenated with preceding text tokens. During inference, single-character generation proceeds autoregressively via greedy decoding until the `<EOS>` token is emitted or the maximum length is reached.

4 Loss Formulation & Optimization

The model is trained by minimizing the character-level Cross-Entropy Loss ($\mathcal{L}_{\text{CE}}$) evaluated across valid non-padded token positions:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{t=1}^T \log P(y_t \mid y_{<t}, Z_v) \quad (3)$$

During baseline training, the loss decays smoothly from 2.81 down to 0.18, with validation loss stabilizing at 0.42, indicating stable convergence without overfitting.

5 Experimental Evaluation

5.1 E0: Overfitting Sanity Check

To verify backpropagation and gradient integrity, experiment *E0* was executed on a micro-batch ($N = 10$ samples). As recorded in `e0_overfit_losses.json`, the training loss rapidly decays from over 3.20 to final values between 0.0035 and 0.0091. This near-zero convergence validates the correctness of custom attention operations, causal masks, and loss calculations.

5.2 Baseline Model Performance

Quantitative evaluation results on the held-out test split are summarized in Table 1.

Table 1: Performance Summary Across Experiments

Experiment	Configuration	Metric	Score / Value
E0 Overfit	Micro-batch ($N = 10$)	Final Training Loss ($\mathcal{L}_{CE}$)	0.0035 – 0.0091
Main Baseline	<code>baseline.yaml</code>	Exact Match (test)	59.7%
Main Baseline	<code>baseline.yaml</code>	Size Accuracy	87.6%
Main Baseline	<code>baseline.yaml</code>	Color Accuracy	80.8%
Main Baseline	<code>baseline.yaml</code>	Shape Accuracy	80.4%
Main Baseline	<code>baseline.yaml</code>	Relation Accuracy	41.7%
E1 Blind	<code>blind.yaml</code>	Exact Match (test)	1.8%

The model achieves high accuracy on individual object attributes (Size: 87.6%, Color: 80.8%, Shape: 80.4%). Performance on spatial relations (41.7%) reflects the higher visual complexity of modeling relative positions.

5.3 E1: Blind Baseline Ablation Study

In experiment *E1* (`configs/blind.yaml`), input image tensors are zeroed out. Exact Match collapses from **59.7%** to **1.8%**, proving that textual output generation is directly grounded in visual inputs.

6 S1: Throughput Benchmark & Hardware Scaling

System execution speed was benchmarked across batch sizes $B \in \{1, 16, 64, 256\}$ on CPU (Intel) and GPU (NVIDIA Tesla T4).

Table 2: Throughput (*S1*) Benchmark Comparison (images / second)

Batch Size (B)	CPU (Intel)	GPU (Tesla T4)
1	22.3 ± 3.2	93.2 ± 0.4
16	56.2 ± 9.8	1381.9 ± 57.4
64	52.4 ± 2.9	2459.8 ± 5.0
256	47.0 ± 0.4	2629.5 ± 6.1

CPU throughput saturates around 52–56 img/s. GPU acceleration reaches **2459.8 img/s** at $B = 64$, yielding a **47 $\times$ speedup** over CPU execution.

7 Software Verification & AI Transparency

7.1 Unit Testing Suite

Code validity was confirmed via 41 unit tests in `tests/`:

- `tests/test_attention.py` (22 tests): Verifies custom MHA math, causal/padding masks, and numerical equivalence with PyTorch scaled dot-product attention.
- `tests/test_shapes.py` (19 tests): Confirms shape transformations across encoder, adapter, and decoder modules.

7.2 AI Usage Compliance

AI tool usage was documented in `AI_USAGE.md`: Claude and Gemini were consulted strictly for environment debugging, pytest execution verification, and LaTeX formatting. All network architectures and custom attention logic were implemented and tested independently.

8 Conclusion

This report demonstrates a functional, lightweight VLM. Overfitting convergence (*E0*), visual grounding (*E1*), and throughput scaling (*S1*) validate the architecture and implementation.